Fysikk-informerte nevrale nettverk

Andrine Holen, Selma Holmen Berg, Emily Ann Mercer

November 10, 2026

1 Generering av syntetiske data

1.1 Oppgave 3.1

Matrisen A og vektoren b^k kan utledes ved å diskretisere varmelikningen ved hjelp av implisitt Euler. Hvor raskt temperaturen T endres over tid approksimeres med implisitt Euler, og Laplace-operatoren er approksimert med differanser i både x- og y-retning. Alle romlige ledd evalueres ved tidssteg $k+1$, noe som gjør metoden implisitt. Ved bruk av denne diskretiseringen kan første- og andreordens deriverte erstattes med approksimasjoner. Som følge av dette kan varmeligningen som oppgitt i ligning (1) omskrives til uttrykket i ligning (2).

$$\frac{\partial T(x,y,t)}{\partial t} = \alpha \Delta T(x,y,t) + q(x,y) \tag{1}$$

$$\frac{T_{i,j}^{k+1} - T_{i,j}^{k}}{\Delta t} = \alpha \left(\frac{T_{i+1,j}^{k+1} - 2T_{i,j}^{k+1} + T_{i-1,j}^{k+1}}{(\Delta x)^2} + \frac{T_{i,j+1}^{k+1} - 2T_{i,j}^{k+1} + T_{i,j-1}^{k+1}}{(\Delta y)^2} \right) + q(x_i, y_j) \tag{2}$$

For å forenkle uttrykket multipliseres begge sidene med Δt for å eliminere brøken på venstre side.

$$T_{i,j}^{k+1} - T_{i,j}^{k} = \alpha \Delta t \left(\frac{T_{i+1,j}^{k+1} - 2T_{i,j}^{k+1} + T_{i-1,j}^{k+1}}{(\Delta x)^2} + \frac{T_{i,j+1}^{k+1} - 2T_{i,j}^{k+1} + T_{i,j-1}^{k+1}}{(\Delta y)^2} \right) + \Delta t \, q(x_i, y_j) \tag{3}$$

Deretter innføres de dimensjonsløse parameterne r_x og r_y for å gjøre mellomregningene mer oversiktlige.

$$r_x = \frac{\alpha \Delta t}{(\Delta x)^2}, \qquad r_y = \frac{\alpha \Delta t}{(\Delta y)^2} \tag{4}$$

Innsettingen av r_x og r_y resulterer i at uttrykket kan skrives på en mer kompakt form.

$$T_{i,j}^{k+1} - T_{i,j}^{k} = r_x(T_{i+1,j}^{k+1} - 2T_{i,j}^{k+1} + T_{i-1,j}^{k+1}) + r_y(T_{i,j+1}^{k+1} - 2T_{i,j}^{k+1} + T_{i,j-1}^{k+1}) + \Delta t \, q(x_i, y_j) \tag{5}$$

Til slutt flyttes alle ledd som inneholder tidssteg $k+1$ til venstre side, og de resterende leddene til høyre.

$$T_{i,j}^{k+1}(1 + 2r_x + 2r_y) - r_x(T_{i+1,j}^{k+1} + T_{i-1,j}^{k+1}) - r_y(T_{i,j+1}^{k+1} + T_{i,j-1}^{k+1}) = T_{i,j}^{k} + \Delta t \, q(x_i, y_j) \tag{6}$$

Ligning (6) representerer én rad i det lineære systemet. Ved å skrive opp tilsvarende ligninger for alle andre gitterpunkt uttrykkes systemet på matriseform som i ligning (7)

$$AT^{k+1} = b^k. \tag{7}$$

der T^{k+1} er en vektorisering av temperaturfeltet ved tidssteg $k+1$.

Matrisen A kommer direkte fra koeffisientene i likning (6). Antall ukjente i systemet er $n = N_x \cdot N_y$, og matrisen $A \in \mathbb{R}^{n \times n}$ lages ved å vektorisere temperaturfeltet på rutenettet slik at hvert

gitterpunkt (i, j) tilsvarer indeksen $i \cdot N_y + j$ og dermed en rad i det lineære systemet. For de indre gitterpunktene består hver rad av fem ikke-null elementer, som følge av fempunktsdiskretiseringen. Hoveddiagonalen har verdi $1 + 2r_x + 2r_y$, mens de to nærmeste naboelementene i x-retning har verdi $-r_x$ og de to nærmeste naboelementene i y-retning har verdi $-r_y$. Alle andre elementer i raden er null. For randpunkter omskrives koeffisientene ut i fra Robin-randbetingelsene. Matrisen A får dermed en karakteristisk båndstruktur.

Randbetingelsene implementeres direkte i matrisen A ved å justere koeffisientene på randen med Robin-randbetingelser. På randen mangler enkelte nabopunkter utenfor domenet, og bidragene fra disse erstattes av ekstra ledd på diagonalen i A sammen med tilsvarende bidrag i høyresiden b^k. Som eksempel betraktes venstre hjørne, der $x = 0$ og normalvektoren er $\mathbf{n} = (-1, 0)$. Randbetingelsen kan dermed skrives som

$$k\frac{\partial T}{\partial x} = h\left(T - T_{\text{out}}\right). \tag{8}$$

Den romlige deriverte kan approksimeres med en endelig differanse,

$$\frac{T_{0,j}^{k+1} - T_{-1,j}^{k+1}}{\Delta x}, \tag{9}$$

som gir

$$k\frac{T_{0,j}^{k+1} - T_{-1,j}^{k+1}}{\Delta x} = h\left(T_{0,j}^{k+1} - T_{\text{out}}\right). \tag{10}$$

Ved å løse dette for det fiktive punktet $T_{-1,j}^{k+1}$, gir det

$$T_{-1,j}^{k+1} = T_{0,j}^{k+1} - \frac{h\Delta x}{k}\left(T_{0,j}^{k+1} - T_{\text{out}}\right). \tag{11}$$

Ved hjelp av ligning (11) kan randbetingelsen implementeres i den diskretiserte varmeligningen ved $i = 0$. Hjelpestørrelse β innføres som vist i ligning (12) for å forenkle uttrykket i ligning (13).

$$\beta = \frac{h\Delta x}{k} \tag{12}$$

$$T_{-1,j}^{k+1} = (1 - \beta)T_{0,j}^{k+1} + \beta T_{\text{out}} \tag{13}$$

Ved å sette uttrykket i ligning (13) inn i den diskretiserte varmeligningen gir det

$$T_{0,j}^{k+1}(1 + 2r_x + 2r_y) - r_x(T_{1,j}^{k+1} + (1 - \beta)T_{0,j}^{k+1} + \beta T_{\text{out}}) - r_y(T_{0,j+1}^{k+1} + T_{0,j-1}^{k+1}) = T_{0,j}^{k} + \Delta t\, q_{0,j}. \tag{14}$$

Deretter kan alle ledd som inneholder tidssteg $k + 1$ samles på venstre side som vil gi uttrykket

$$(1 + 2r_x + 2r_y - r_x(1 - \beta))T_{0,j}^{k+1} - r_x T_{1,j}^{k+1} - r_y(T_{0,j+1}^{k+1} + T_{0,j-1}^{k+1}) = T_{0,j}^{k} + \Delta t\, q_{0,j} + r_x \beta T_{out}. \tag{15}$$

Forenkling av koeffisienter gir til slutt uttrykket

$$(1 + r_x + 2r_y + r_x\beta)T_{0,j}^{k+1} - r_x T_{1,j}^{k+1} - r_y(T_{0,j+1}^{k+1} + T_{0,j-1}^{k+1}) = T_{0,j}^{k} + \Delta t\, q_{0,j} + r_x \beta T_{out}. \tag{16}$$

De øvrige randene i systemet vil få tilsvarende form. For hvert randpunkt gir dette et ekstra bidrag på diagonalen i matrisen A samt et tilsvarende ledd i høyresiden $\mathbf{b}^k$. Dette samsvarer med implementasjonen i koden. I `_build_matrix()` legges Robin-bidraget til på diagonalen for randpunktene, mens `_build_rhs()` legger til et tilsvarende ledd i høyresiden proporsjonalt med T_{out}.

Høyresiden i systemet, altså vektoren b^k, bygges i funksjonen `_build_rhs()`. Denne tar utgangspunkt i temperaturfeltet fra forrige tidssteg $T_{i,j}^k$, slik som vist på høyresiden i likning (6). I tillegg legges bidraget fra varmekilden q til, samt ekstra ledd som følger av Robin-randbetingelsene. Til slutt flates høyresiden ut til en vektor som kan brukes direkte i løsningen av det lineære systemet.

1.2 Oppgave 3.4

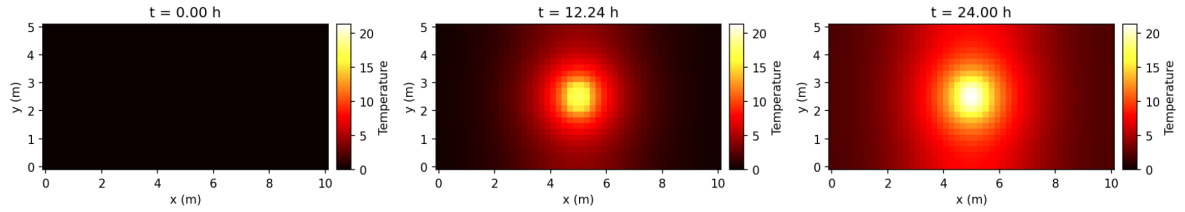


Figure 1: Temperaturutvikling for den numeriske løsningen over 24 timer. Animasjonen viser en jevn spredning fra varmekilden.

Den numeriske modellen gir et bilde av temperaturfordelingen i rommet slik som vist i figur 1. Modellen kan brukes som et effektivt verktøy for å redusere energiforbruket ved oppvarming av boliger. Ved å kombinere modellen med sensordata er det mulig å studere hvordan varme fordeler seg i et rom både med hensyn til tid og posisjon, og dermed identifisere hvor varmetap faktisk oppstår. Modellen gjør det mulig å undersøke effekten av isolasjon ved å sammenligne temperaturutvikling og varmetap for ulike randbetingelser. Dersom simuleringene viser at temperaturen faller raskt nær yttervegger eller vinduer, kan dette indikere dårlig isolasjon. Bedre isolasjon reduserer varmetapet og gjør at varmekilden kan operere med lavere effekt for å opprettholde ønsket innendørstemperatur, noe som kan føre til lavere energiforbruk og reduserte kostnader.

Videre kan modellen brukes til å optimalisere plasseringen av varmekilder. I denne simuleringen er varmekildens styrke P satt til $15\,\mathrm{K\,s^{-1}}$ i filen `config.yaml`. Varmeoverføringskoeffisienten $h = 0.04\,\mathrm{W\,m^{-2}\,K^{-1}}$, varmeledningsevnen $k = 0.0\,\mathrm{W\,m^{-1}\,K^{-1}}$, den termiske diffusiviteten $\alpha = 0.25\,\mathrm{m^2\,s^{-1}}$ og utetemperaturen er $T_{\mathrm{out}} = 0.0\,^{\circ}\mathrm{C}$. Såpass lave verdier for blant annet varmetap til omgivelsene, k, er urealistiske hvis man henter verdiene fra sensordata. De fysiske parameterne bestemmer hvor mye varme som tilføres rommet, og hvor raskt varme tapes gjennom randene. Når temperaturen øker, vil også varmetapet øke. For disse innstillingene når temperaturen et maksimum rundt $20\,^{\circ}\mathrm{C}$ i området nær varmekilden.

Ved å simulere ulike plasseringer av varmekilder kan man undersøke hvordan varmen fordeles i rommet, og identifisere områder som blir overopphetet eller for kalde. Dette gir et bedre grunnlag for å vurdere om varmekilder bør flyttes eller om effekten bør justeres for å oppnå en jevnere temperaturfordeling. Denne teorien kan testes ved å legge til flere varmekilder i simuleringen. Ved å legge til en varmekilde i ett av rommets hjørner, slik som koden la opp til, ble det spesifikke området varmet opp raskere, men det hadde liten effekt for resten av rommet. Figur 2 viser temperaturfordelingen med fire ekstra sensorer plassert jevnt rundt i rommet. Her blir rommet varmet opp til omtrent 20 grader. Såpass mange varmekilder i et rom er åpenbart mer kostbart og krever mer energi, men det har stor effekt. Andre løsninger kunne vært to større kilder plassert ved siden av hverandre, eller mange små.

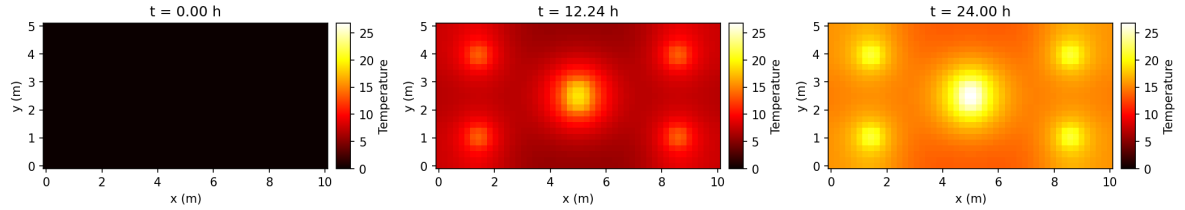


Figure 2: Temperaturfordeling for den numeriske løseren med fire ekstra varmekilder plassert jevnt i rommet. Animasjonen viser varmen er godt utspredt.

I tillegg kan modellen brukes til å vurdere effektiviteten av varmekilder for å gjøre en avveining

mellom komfort og energibruk. Dersom simuleringene viser at ønsket temperatur kun oppnås ved høy tilført effekt, eller at deler av rommet forblir kalde over tid, kan det indikere at varmekilden er underdimensjonert eller ineffektiv. Modellen kan dermed brukes til å vurdere alternative løsninger. Dette kan være endret plassering av varmekilden, bruk av flere varmekilder, eller overgang til mer energieffektive systemer, som for eksempel varmepumpe.

Ved å simulere ulike situasjoner er det mulig å finne en balanse mellom jevn temperaturfordeling og lavt energiforbruk. Kombinert med sanntids sensordata, kan modellen inngå i et intelligent system der varmeeffekten justeres dynamisk etter utetemperatur, bruksmønster og ønskede innstillinger. Dette er særlig relevant nå som klimaet fører til store temperaturvariasjoner. Faste innstillinger kan føre til unødvendig energibruk. Ettersom oppvarming av bygg i Norge stod for nær en tredjedel av det totale energiforbruket i 2021 [6], vil slike sensorstyrte systemer kunne åpne opp for store energibesparelser.

1.3 Oppgave 3.5

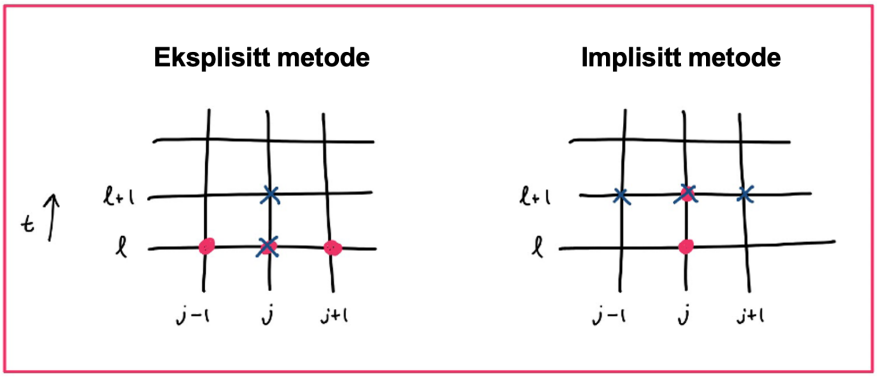


Figure 3: Illustrasjon av eksplisitt og implisitt tidsdiskretisering. Rosa punkter markerer verdier som er kjent fra forrige tidssteg. I den eksplisitte metoden beregnes verdien i neste tidssteg direkte fra disse kjente punktene. I den implisitte metoden representerer de blå punktene ukjente verdier på neste tidsnivå, som er koblet sammen og derfor må bestemmes samtidig ved å løse et lineært ligningssystem.

For å løse varmeligningen numerisk kan det brukes både eksplisitte og implisitte metoder. Forskjellen mellom disse ligger i hvordan temperaturfeltet oppdateres over tid, noe som også illustreres i figur 3 . I en eksplisitt metode beregnes temperaturen i neste tidssteg direkte fra kjente verdier fra forrige tidssteg. Som vist i figur 3 benyttes kun verdier fra tidligere steg til å beregne den nye temperaturen. Dette gir en enkel og rask utregning, men metoden er kun stabil hvis tidssteget er tilstrekkelig lite. For varmeligningen innebærer dette strenge stabilitetskrav, som kan føre til behov for mange tidssteg.

I løseren for den numeriske metoden benyttes en implisitt metode; implisitt Euler. Her inngår temperaturene ved neste tidssteg på begge sider av den diskretiserte ligningen. Dette betyr at de ukjente temperaturene i neste tidsnivå er koblet sammen, og må bestemmes samtidig ved å løse det lineære ligningssystemet

$$AT^{k+1} = b^k,$$

for hvert tidssteg.

Fordelen med denne metoden er at den er stabil også for store tidssteg, slik at det kan bruke færre tidssteg totalt. Ulempen er at hvert tidssteg blir mer beregningstungt, siden det krever løsing av et lineært system. I implementasjonen løses dette systemet direkte ved hjelp av `np.linalg.solve()`, som baserer seg på matrise-faktorisering. Dette kan bli kostbart for store systemer, spesielt siden systemmatrisen har størrelse $n \times n$, der $n = N_x \cdot N_y$.

Tiden modellen bruker på å regne hvert steg kan reduseres ved å utnytte strukturen til systemmatrisen. Matrisen A som kommer fra fempunktsdiskretisering av Laplace-operatoren er sparsom, det vil si at hver rad kun inneholder noen få ikke-null elementer, mens resten er null. I praksis betyr dette at hver gitterpunkt kun er koblet til sine nærmeste naboer, slik figur 3 viser. [3, Kap. 3].

Videre kan beregningene effektiviseres ved parallellisering av løseren. I både direkte og iterative metoder består hoveddelen av regnearbeidet av matrise-vektor-operasjoner, hvor hver rad i systemmatrisen bidrar uavhengig til oppdateringen av løsningen. Parallelisering er det som gjør det mulig å utføre beregningene for ulike gitterpunkter samtidig, uten at de er gjensidig avhengige av hverandre i samme iterasjon. [3, Kap. 6–7]

I praksis betyr dette at arbeidet kan fordeles over flere prosessorkjerner, der hver kjerne håndterer deler av matrisen eller bestemte områder av rutenettet. Iterative metoder som Conjugate Gradient er spesielt godt egnet for parallellisering, siden den i stor grad består av gjentatte matrise-vektor-multiplikasjoner og indre produkter, som kan utføres parallelt [3, Kap. 6–7]. Dette gjør det mulig å redusere kjøretiden for store systemer, spesielt på moderne maskinvare med flere kjerner eller på GPU-er. [1].

1.4 Oppgave 3.6

Dersom de fysiske parameterne α, k og h antas å være ukjente, kan de estimeres ved å kombinere sensordata med den numeriske løseren `solve_heat_equation()`. Dette kan behandles som et optimeringsproblem, der målet er å finne parameterverdier som gir best mulig samsvar mellom modellert- og observert temperatur.

Den mest standardiserte metoden for optimalisering er minste kvadraters metode (17). Ved å velge en rimelig startverdi for parameter α, k og h, kan de innsettes i den numeriske modellen som beregner temperaturutviklingen i rommet. Deretter beregnes avviket mellom den simulerte og observerte temperaturen. Det vil være hensiktsmessig å sette fysiske begrensninger for parametrene, for eksempel at α, k, h skal være positive.

$$J(\alpha, k, h) = \sum_{i=1}^{N_s} \sum_{k=1}^{N_m} \left(T_{\text{modell}}(s_i, t_k; \alpha, k, h) - T_{\text{obs}}(s_i, t_k)\right)^2 . \tag{17}$$

Feilfunksjonen $J(\alpha, k, h)$ minimeres ved hjelp av en iterativ optimeringsmetode. Parameterne α, k og h justeres gradvis for å redusere avviket mellom modellert og observert temperatur. Justeringen kan for eksempel gjøres ved å endre parameterne med en valgt steglengde basert på hvordan feilen utvikler seg mellom iterasjonene. Denne prosessen gjentas til feilen er minimert, eller endringene blir små nok. På denne måten kan de ukjente fysiske parameterne estimeres ved å kombinere sensordata med den numeriske modellen, slik at modellen treffer best mulig på den observerte temperaturutviklingen. Denne typen parameterestimering danner også et naturlig utgangspunkt for mer avanserte metoder, som for eksempel modell- og data-drevne tilnærminger basert på nevrale nettverk. [1].

2 Nevrale nettverk

2.1 Oppgave 4.5

Resultatet fra treningen av det nevrale nettverket ble som fremstilt i figur 4. I den numeriske løsningen antas det at varmekilden sprer varme jevnt, og at veggene har like egenskaper med hensyn til isolasjon, tykkelse og starttemperatur. Dette gir en symmetrisk og fysisk realistisk temperaturfordeling, slik som observert i figur 1. Det nevrale nettverket trenes derimot kun på simulerte sensordata. Simuleringen tar utgangspunkt i et rom på 5x10 meter med totalt 9 sensorer, hvorav én sensor er sentrert ved varmekilden, to på hver langside mellom veggen og varmekilden, og de resterende er plassert symmetrisk langs veggene. Nettverket tar imidlertid ikke hensyn til de fysiske lovene som styrer varmeledning, men optimaliserer kun mot sensordata og initialbetingelser.

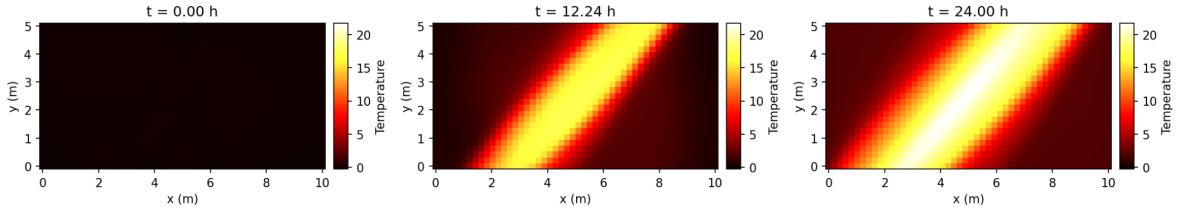


Figure 4: Temperaturutvikling for det nevrale nettverket over 24 timer. Animasjonen viser et predikert diagonalt varmt område i rommet.

Ved å sammenligne den numeriske løseren i figur 1 med det nevrale nettverket i figur 4, blir det tydelig at det nevrale nettverket i begrenset grad er i stand til å lære den faktiske temperaturfordelingen i rommet fra sensordata alene. I stedet for å vise en jevn og symmetrisk spredning fra varmekilden, utvikler det nevrale nettverket en mindre realistisk temperaturfordeling som vises som et diagonalt oppvarmet område i plottet. Dette diagonale området kan skyldes at nettverket forsøker å sette opp en fordeling mellom sensordataene uten noen fysisk forståelse av varmeutbredelse, og finner derfor sammenhenger som minimerer tapet, uten at de gjenspeiler fysiske lover. Avviket fra den numeriske modellen kommer tydelig frem i figur 5, som viser differansen mellom den numeriske løsningen og nettverkets prediksjon. Figuren viser at langs det diagonale området er NN-modellen opptil 15 grader varmere enn i den numeriske modellen, noe som tyder på et stort avvik. Dette kunne enkelt fremstilles ved å legge til koden fra listing 1 inn i filen `run_nn.py`.

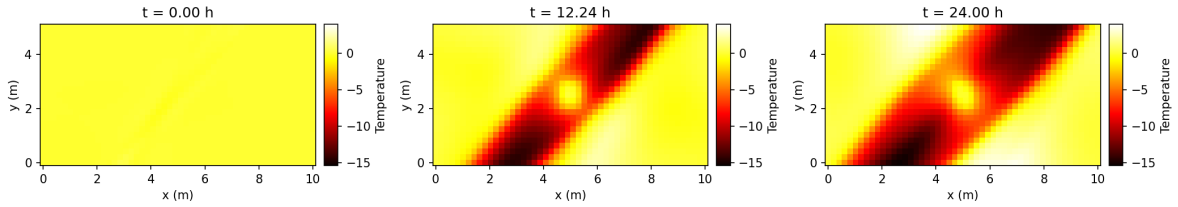


Figure 5: Differanse temperaturfordeling mellom numerisk løser og nevralt nettverk. Det observeres store forskjeller i temperatur langs diagonalen. Animasjonen viser et avvik på opp mot 15 grader.

Listing 1: Plotting av differanse mellom numerisk løser og NN-prediksjon

```
plot_snapshots(x, y, t, T_fdm - T_pred,
               save_path="output/nn/nn_difference_snapshots.png")

create_animation(x, y, t, T_fdm - T_pred,
                 title="NN",
                 save_path="output/nn/nn_difference_animation.gif")
```

For å estimere de ulike parameterne i det nevrale nettverket defineres en objektivfunksjon. Denne kan bestå av flere ulike komponenter. Under treningen av NN er objektivet at forskjellen mellom de predikerte og observerte verdiene skal være minimal, noe som løses gjennom å minimere tapsfunksjoner. For det nevrale nettverket er det definert to ulike tapsfunksjoner; en for sensordata og en for initialbetingelsene.

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_s \cdot N_m} \sum_{i=1}^{N_s} \sum_{k=1}^{N_m} (f_\theta(s_i, t_k) - T_{obs}^{i,k})^2 \tag{18}$$

$$\mathcal{L}_{\text{ic}}(\theta) = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} (f_\theta(s_i^{ic}, 0) - T_{out})^2 \tag{19}$$

I ligning (18) er N_s antall sensorer og N_m antall målepunkter. Med gjennomsnittlig kvadratiske feil minimeres tapet med nettverkets predikerte temperatur, f_θ, og observert temperatur T_{obs}. Tapsfunksjonen for initialbetingelsene er gjengitt i ligning (19). N_{ic} angir antall punkter initialbetingelsen evalueres i, og s_{ic} angir romlig posisjon ved tid $t = 0$. Tapsfunksjonen sammenligner predikert temperatur med kjent initialtemperatur T_{out}. De to nevnte tapskomponentene utgjør objektivfunksjonen (20).

$$\mathcal{L}(\theta) = \lambda_{data}\mathcal{L}_{data}(\theta) + \lambda_{ic}\mathcal{L}_{ic}(\theta) \tag{20}$$

Her uttrykkes det totale tapet $\mathcal{L}(\theta)$ som summen av komponentene $\mathcal{L}_{\text{data}}(\theta)$ og $\mathcal{L}_{\text{ic}}(\theta)$ sammen med deres respektive vekter, λ_{data} og λ_{ic}. Vektene skalerer gradientbidraget til de to tapskomponentene, og vil påvirke hvordan parameterne oppdateres under trening. Minimeringen av tapet under treningen av det nevrale nettverket vises i figur 6.

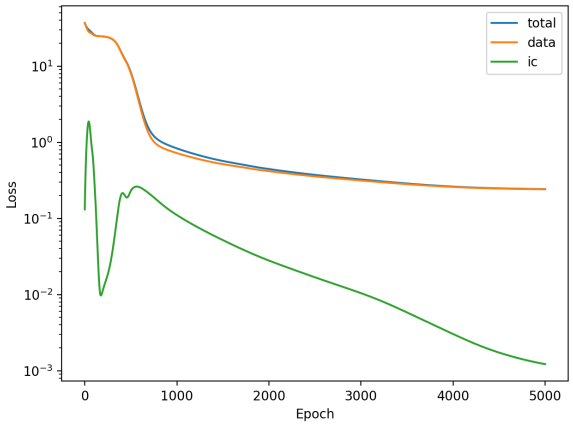


Figure 6: Viser minimering av tap for sensordata og initialbetingelser over 5000 epoker for et nevralt nettverk.

Starten av plottet viser et kraftig fall i tapet, noe som indikerer at det nevrale nettverket raskt korrigerer store feil og lærer de mest åpenbare mønstrene i dataene. For "*data*" konvergerer funksjonen relativt tidlig, rundt epoke 1000, noe som tyder på at nettverket tidlig har tilpasset seg sensordataene og temperaturprediksjonen. "*IC*" fortsetter derimot å synke nesten lineært gjennom hele treningsperioden, noe som viser at modellen fortsetter å tilpasse seg initialbetingelsene. Til tross for dette tyder temperaturutviklingen for NN i figur 6 på at tilpassingen av initialbetingelsene har liten innvirkning på totale tapet, som i hovedsak følger utviklingen til tapsfunksjonen for sensordata.

Antall sensorer og deres plassering har stor betydning for hvor godt nettverket kan lære temperaturfeltet. Dette kan forklare den ujevne temperaturfordelingen for det nevrale nettverket i figur 4. For å undersøke denne avhengigheten viser figur 7 temperaturfordelingen når det plasseres én sensor per kvadratmeter i rommet. I dette tilfellet klarer nettverket i langt større grad å gjenskape et realistisk temperaturfelt, noe som illustrerer at NN kan fungere godt dersom datagrunnlaget er tilstrekkelig rikt og jevnt fordelt. Det må derimot betydelig mange sensorer til for at temperaturfordelingen skal bli realistisk.

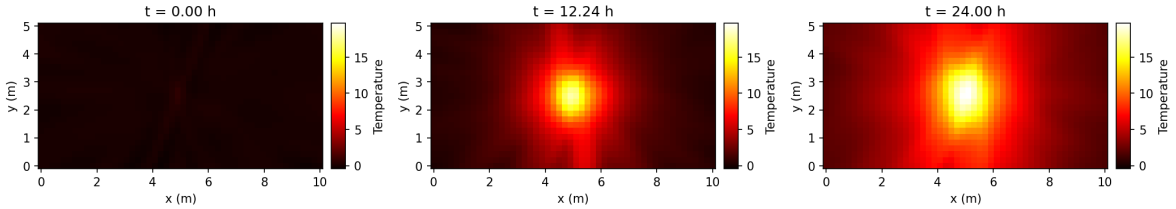


Figure 7: Temperaturfordelingen for det nevrale nettverket med én sensor per kvadratmeter

Ettersom nettverket kun predikerer temperaturfeltet med sensordata, har mengden støy stor innvirkning på tapene. Standardverdien for støy under treningen av det nevrale nettverket er satt lik 0.5. Ved å doble mengden støy til verdi 1.0, øker tapene for $\mathcal{L}_{\text{data}}$ som forventet. Sensordata er mer inkonsistent og det vil være vanskeligere for systemet å tilpasse seg. Med støyparameteren satt til 0.001 minker tapene drastisk. Dette leder også til at det totale tapet blir merkbart lavere, i tillegg til at $\mathcal{L}_{\text{ic}}$ har en liten nedgang. Figur 8 visualiserer hvor stor innvirkning støymengden har på tapene til det nevrale nettverket. Tapet for initialbetingelsen har liten endring i sammenheng med variasjon av støy. Dette er fordi initialbetingelsene er deterministiske, og dermed uavhengig av støy i sensordataene.

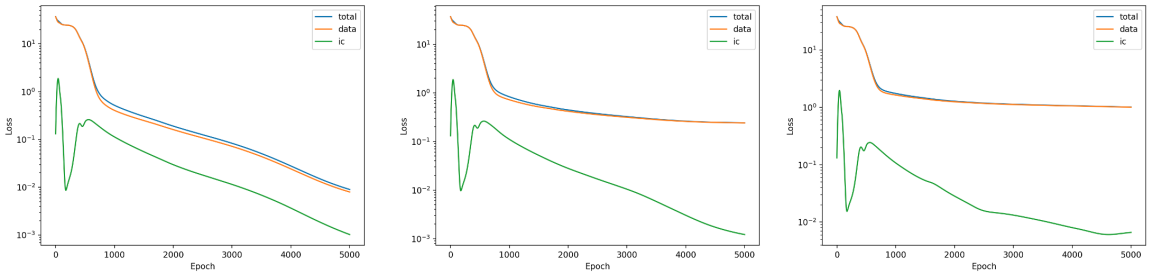


Figure 8: Viser tapsfunksjonen for det nevrale nettverket ved varierende støyparameter. Støyparameter satt lik 0.001 i plott til venstre, standardverdi på 0.5 i midten, og 1.0 i høyre plott.

Til tross for at tapene øker betyr det ikke nødvendigvis at minst mulig støy er ekvivalent med den mest optimale prediksjonen. I figur 9 vises temperaturutviklingen i rommet når støyparameteren økes fra verdien 0.001 til 1.0. Et lavt støynivå gir konsistente sensordata, noe som gjør at nettverket lærer presise romlige korrelasjoner. Varmetransporten i data har en foretrukket retning, og nettverket vil forsterke dette. Dette fenomenet resulterer i et skarpt diagonalt bånd slik som observert for de to animasjonene med lavest støyparameter. For at det nevrale nettverket ikke skal ta matematiske snarveier som gir virkelighetsfjerne prediksjoner, kan en moderat mengde støy bidra til å regulere prediksjonene. Blir det derimot for mye støy, kan det lede til at modellen tilpasses tilfeldige variasjoner, noe som gir høy varians. Gradientinformasjonen i x- og y-retning blir upålitelig, og nettverket vil ikke være i stand til å identifisere en tydelig retning, slik som observert for animasjonen med høyest støyparameter.

Resultatene av nettverkets prediksjoner avhenger videre av nettverksarkitekturen. Antall skjulte lag angir hvor komplekse strukturer nettverket kan lære, mens antall nevroner per lag angir hvor detaljerte disse strukturene er. For få lag og nevroner vil gi et svakt nettverk med høyt datatap, som vises i grafen til venstre i figur 10. Nettverket vil kun lære en grov tilnærming som gir en begrenset uttrykksevne. Flere lag gir mer komplekse systemer og mer fleksibilitet. I plottet for tapsfunksjonen til høyre med flest lag og dobbelt så mange nevroner per lag, observeres det spisser fra rundt epoke 2000 for initialbetingelsene. Dette kan skyldes at modellen periodevis bryter med initialbetingelsene for å optimere datatapet. De vil deretter korrigeres som skaper oscilasjoner i plottet. Med økende antall lag medfører også en risiko om overtilpasning, spesielt med et begrenset antall sensorer. Siden modellen kun tar inn sensordata, vil for mange lag og nevroner resultere i at systemet lærer urealistiske mønstre

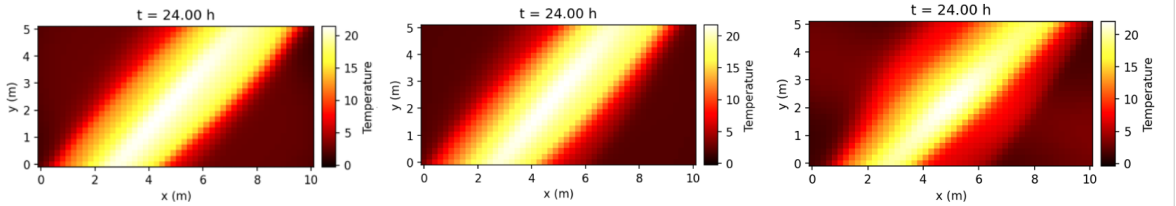


Figure 9: Temperaturutvikling etter 24 timer med økende støyparameter fra venstre til høyre. Støyparameter satt lik 0.001 i plott til venstre, standardverdi lik 0.5 i midten, og 1.0 i høyre plott.

og tilpasser seg støy i stedet for temperaturfordelingen. Dette mønsteret kan observeres i figur 11. De diagonale strukturene vil altså indikere en overtilpasning grunnet mer detaljerte, men mindre fysiske temperaturmønstre.

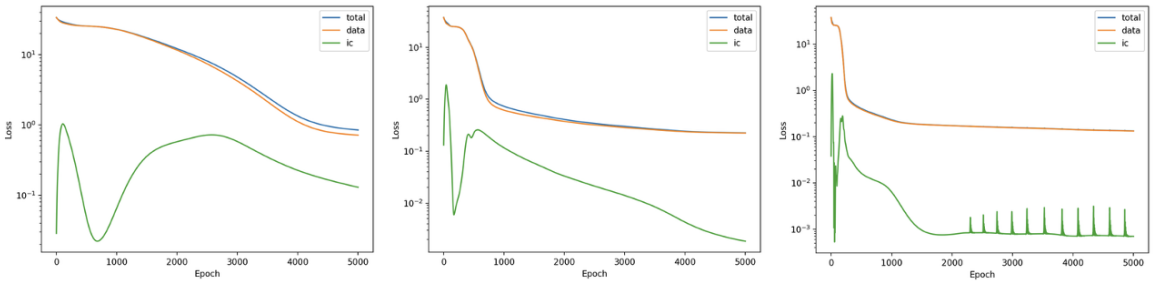


Figure 10: Viser tapsfunksjonene ved varierende antall lag, og nevroner per lag. Figuren til venstre har et lag med 16 nevroner, figuren til midten er utgangspunktet med to lag med 32 nevroner, og figuren til høyre har tre lag med 64 nevroner.

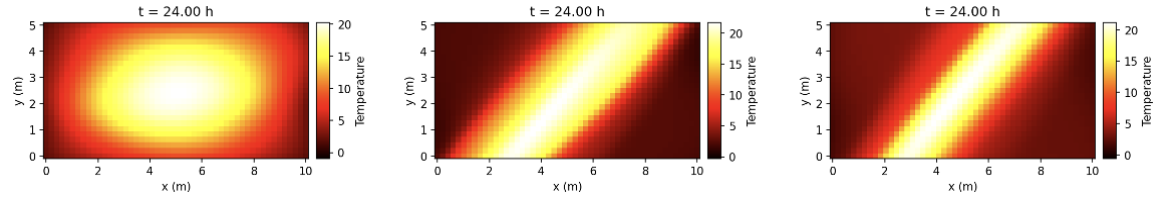


Figure 11: Viser temperaturfordelingen etter 24 timer ved varierende antall lag, og nevroner per lag. Figuren til venstre har ett lag med 16 nevroner, figuren i midten er utgangspunktet med to lag med 32 nevroner, og figuren til høyre har tre lag med 64 nevroner.

I tillegg til nettverksarkitekturen avhenger kvaliteten av modellen i stor grad av treningen. Antall epoker styrer hvor mange runder det nevrale nettverket oppdaterer parameterne. I en modell med for få epoker risikerer dette at modellen vil være undertrent. Med for få epoker kan det føre til at det ikke oppnås en tydelig konvergens for tapsfunksjonene, og at den ikke får oppfylt initialbetingelsene slik som avbildet til venstre i figur 12. Et svært høyt antall epoker kan føre til overtilpasning ved at modellen tilpasser seg støy eller andre detaljer fra sensordata. Som tidligere nevnt, illustrerer figur 6 at tapsfunksjonen for sensordata begynner å konvergere etter rundt 1000 epoker, noe som indikerer at modellen raskt utnytter informasjonen i sensordata. Tapet for initialbetingelsen derimot fortsetter å avta over flere epoker. Dette viser at modellen fortsetter å gjøre små justeringer for å tilfredsstille initialbetingelsene uten at det har en betydelig innvirkning på det totale tapet. Økning av epoker kan

ikke kompensere for begrensninger i data. Når det totale tapet har konvergert, vil den videre treningen gi marginal gevinst.

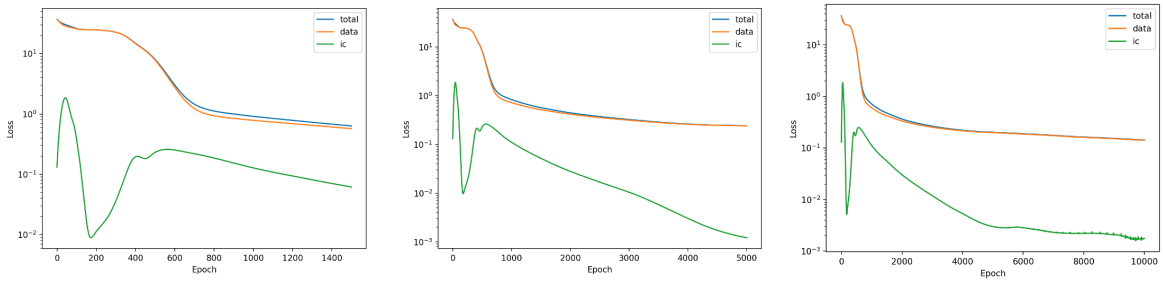


Figure 12: Tapsfunksjoner for nevralt nettverk med økende antall epoker. Antall epoker i grafen til venstre er satt lik 1500, grafen i midten har standardmengden på 5000 epoker, og grafen til høyre har 10000 epoker.

Når tapsfunksjonen har konvergert, vil forbedringene avhenge av optimaliseringen og valgene av parametre til den stokastiske optimeringsalgoritmen "Adam". Valget av parametere i Adam-algoritmen har stor betydning for både stabiliteten og hastigheten på treningen. Læringsraten bestemmer hvor store steg som tas i parameterrommet når nettverket oppdateres. Under treningen av det nevrale nettverket var læringsraten satt til verdien 0.001. En for høy læringsrate kan føre til ustabil trening med store svingninger i tapet, slik som observert i figur 13. De store spissene viser at optimaliseringen ikke forbedrer seg, altså blir det numerisk ustabilitet uten forbedret læring. Når læringsraten er for lav resulterer det i langsom konvergens som observert i figur 14. Tapene for initialbetingelsene har ingen svingninger slik som for den høye læringsraten, og vil etter et stort fall øke og flate ut. Dette resulterer i en bedre generalisering, men krever mer trening og flere epoker ettersom tapsfunksjonene ikke har tydelig konvergert etter 5000 epoker i figur 14.

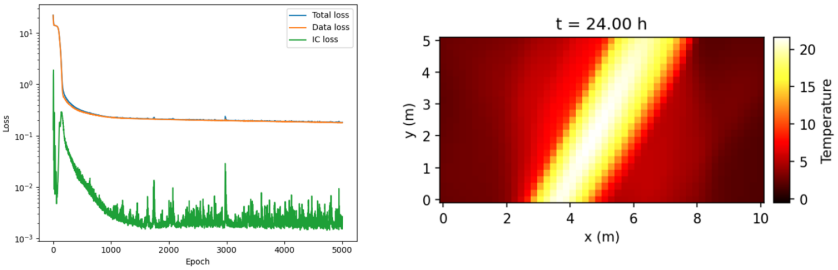


Figure 13: Viser tapsfunksjonen og temperaturfordelingen til det nevrale nettverket når læringsraten er økt til 0.01. FIgurene indikerer mer numerisk ustabilitet.

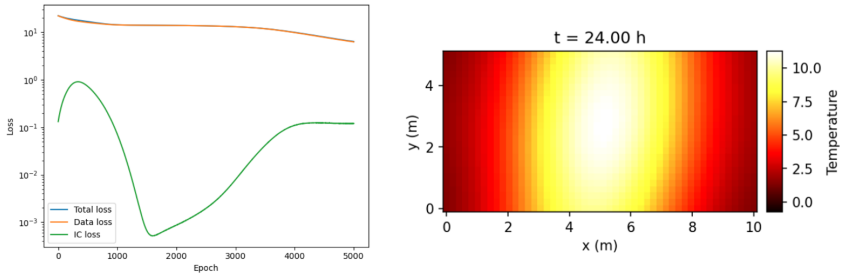


Figure 14: Viser tapsfunksjonen og temperaturfordelingen til det nevrale nettverket når læringsraten er minket til 0.0001. Figurene indikerer bedre generalisering og tregere konvergens.

3 Fysikk-informerte nevrale nettverk

3.1 Oppgave 5.5

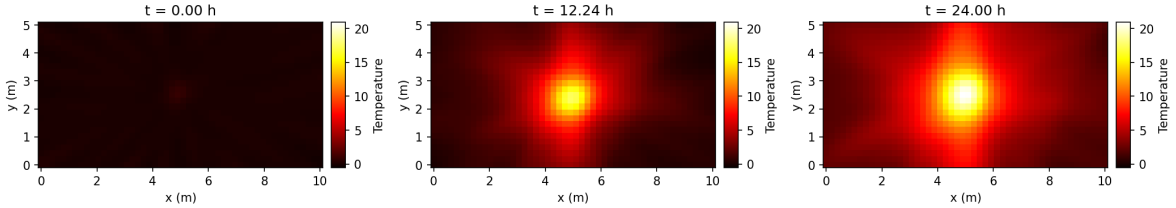


Figure 15: Temperaturutvikling over 24 timer for det fysikk-informerte nevrale nettverket, PINN

Resultatene fra det fysikk-informerte nevrale nettverket (PINN) er vist i figur 15. Sammenlignet med den numeriske løsningen vises det at PINN gir en jevn og relativt symmetrisk temperaturfordeling, der varmen sprer seg utover fra varmekilden i midten av rommet på en fysisk realistisk måte. Dette står i tydelig kontrast til det rene nevrale nettverket (NN), som utvikler en mindre realistisk temperaturfordeling med et diagonalt varmt område, vist i figur 4. Denne diagonale stripen oppstår fordi NN kun trenes på sensordata og initialbetingelser, og dermed mangler informasjon om de fysiske lovene som styrer varmeledning i rommet. PINN fungerer dermed som en bedre approksimasjon av den numeriske løsningen enn NN, selv om den ikke er identisk eller perfekt symmetrisk. Avvikene kan forklares av støy i sensordataene og usikkerhet i de estimerte fysiske parameterne, ettersom PINN må lære disse fra data under treningen, mens den numeriske løseren benytter kjente og eksakte parameterverdier.

For å tydeliggjøre forskjellene mellom de tre modellene, er temperaturdifferanser visualisert i figurene nedenfor. Til venstre i figur 16 vises differansen mellom FDM-løsningen og PINN-prediksjonen. Avvikene i temperaturfordelingen ligger hovedsakelig mellom -1 og 2 grader, noe som indikerer at PINN gir en god tilnærming til den numeriske løsningen. Store deler av området har små avvik, mens større forskjeller kan knyttes til områder der sensordataene er mangelfulle, eller der parameterestimatene har stor usikkerhet. Dette er forventet, ettersom FDM-løseren baserer seg på kjente og eksakte fysiske parametere, mens PINN-en må estimere disse fra data under treningen. Til høyre i figur 16 vises differansen mellom NN- og PINN-modellene. Her er temperaturdifferansen nær null i store deler av rommet, men med tydelige avvik langs diagonalen, som kommer fra den urealistiske temperaturfordelingen i NN. Differansen i temperatur kunne fremstilles ved å legge inn kodene fra listing 2 og 3 i fila `run_pinn`.

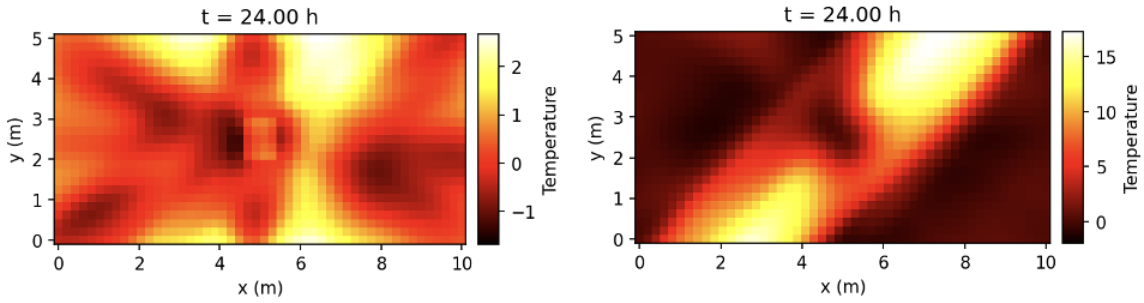


Figure 16: Temperaturfordelingen til venstre viser differansen mellom numerisk løser og PINN, mens den til høyre viser differanse mellom NN og PINN.

Listing 2: Plotting av differanse mellom numerisk løser og PINN-prediksjon

```
plot_snapshots(x, y, t, T_fdm - T_pred_pinn,
               save_path="output/nn/nn_difference_snapshots.png")

create_animation(x, y, t, T_fdm - T_pred_pinn,
                 title="NN",
                 save_path="output/nn/nn_difference_animation.gif")
```

Listing 3: Plotting av differanse mellom NN og PINN-prediksjon

```
plot_snapshots(x, y, t, T_pred_nn - T_pred_pinn,
               save_path="output/nn/nn_difference_snapshots.png")

create_animation(x, y, t, T_pred_nn - T_pred_pinn,
                 title="NN",
                 save_path="output/nn/nn_difference_animation.gif")
```

Når det gjelder antall sensorer og deres plassering, påvirkes både NN og PINN av dette, men på ulike måter. Som tidligere nevnt er NN sterkt avhengig av mange og godt fordelte sensorer for å kunne gi realistiske prediksjoner. For å rekonstruere temperaturfeltet langs veggene må nettverket i teorien ha gode målinger i disse områdene. Dette illustreres i figur 7, der en sensor per kvadratmeter gir en langt mer symmetrisk og realistisk temperaturfordeling som ligner den numeriske løsningen. Antall sensorer og deres plassering er altså avgjørende for hvor godt NN klarer å tolke rommet.

PINN klarer i større grad kompensere for manglende eller dårlig plasserte sensorer ved hjelp av den fysiske modellen. Dette gjør at PINN kan gi rimelige prediksjoner også i områder langt fra sensorene, hvis de for eksempel ikke ligger langs randen. Dette observeres tydelig i figur 15, som er basert på samme sensordata som NN-modellen, men likevel gir en jevn temperaturfordeling som følge av fysikkleddet, initial- og randbetingelsene. Samtidig viser figur 17 at også PINN forbedres av bedre sensordekning. Når antall sensorer økes til en sensor per kvadratmeter, blir temperaturfordelingen nesten identisk med den numeriske løsningen. Bedre sensordata gir altså bedre resultater for begge metodene, men selv med begrensede data klarer PINN å tolke rommet og temperaturfordelingen bedre enn NN.

I likhet med NN gir økt støynivå en økning i tap for $\mathcal{L}_{data}$ som observert i figur 18. Det observeres ikke store forskjeller i temperaturspredningen for støyparameter lik 0.001 og 0.5. Dette tyder på at fysikkleddet sammen med rand- og initialbetingelsene fortsatt dominerer. Når støyparameteren dobles til verdi 1.0 blir avvikene større, noe som gir en mer ujevn varmefordeling som vist i figur 19. PINN klarer ikke lenger å tilfredsstille fysikken og sensordata, og gjør dermed tilpasninger til støyet som gir de observerte avvikene.

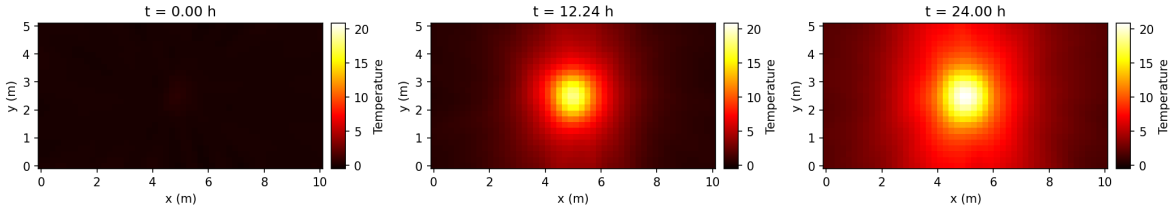


Figure 17: Figuren viser temperaturfordelingen over 24 timer, for PINN med en sensor per kvadratmeter

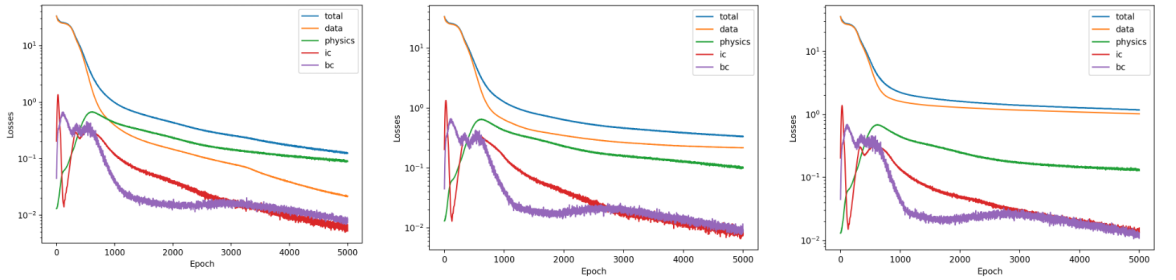


Figure 18: Tapsfunksjon for PINN med økende støy. Støyparameter satt lik 0.001 til venstre, standardverdi 0.5 i midten, og 1.0 til høyre. Viser at $\mathcal{L}_{data}$ øker med støyparameteren, noe som har stor innvirkning på det totale tapet.

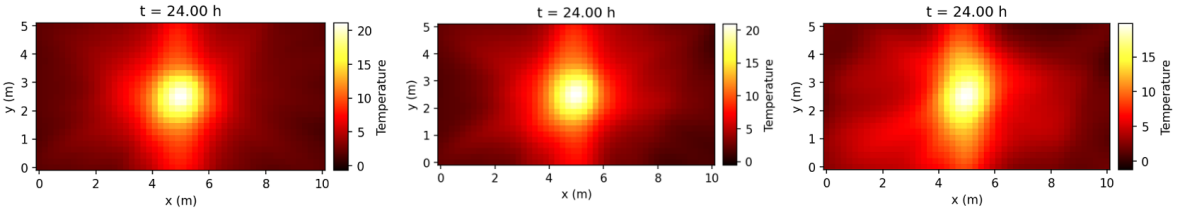


Figure 19: Temperaturutvikling over 24 timer med økende støyparameter fra høyre til venstre. Støyparameter satt lik 0.001 til venstre, standardverdi 0.5 i midten, og 1.0 til høyre. Viser at fysikkleddene kombinert med rand- og initialbetingelsene dominerer treningen intill verdien for støyparamteren blir for høy som observert til høyre.

I sammenligning med det nevrale nettverket, påvirkes de to metodene ulikt av støynivå. For NN behandles all sensordata likt, uten å ta hensyn til fysiske lover. Dette kan føre til en overforenklet sammenheng mellom temperatur og sensormåling for lavt støynivå, der modellen risikerer å tilpasse seg tilfeldige variasjoner for høye støynivåer. PINN vil påvirkes i mindre grad ettersom fysikkleddet og betingelsene fungerer som en form for regulering som demper effekten av støy. I tillegg til å tilpasse seg sensordata, må PINN minimere PDE-residualene og oppfylle både initial- og randbetingelsene, noe som presser løsningen mot et glattere og mer fysisk konsistent temperaturfelt. Graden av denne støydempingen kan styres gjennom prioriteringen i tapsfunksjonen, spesielt ved justering av $\mathcal{L}_{data}$, som balanserer innflytelsen fra sensordata mot de fysiske leddene. En PINN vil altså være mer motstående til støydata, sammenlignet med et sensorbasert nevralt nettverk.

Videre ble de to modellene ulikt påvirket av å endre antall epoker. Figur 12 indikerer at NN var lite påvirket av et økt antall epoker, fordi den kun fortsatte å tilpasse seg initialbetingelsene etter at datatapsfunksjonen begynte å konvergere. Siden NN ikke inneholder noen fysisk regularisering, gir dette kun

små endringer i det globale temperaturfeltet, og flere epoker fører derfor ikke til vesentlig forbedring av resultatene. Det kan derimot føre til overtilpasning. For PINN derimot fortsetter modellen å tilpasse seg både fysikk-, randbetingelses-, og initialbetingelses-tapfunksjonene etter datatapsfunksjonen konvergerer. Dersom nettverket trenes over for eksempel 10 000 epoker, vil nettverket tilpasse seg disse parametrene for godt, noe som resulterer i en temperaturfordeling som vist i figur 20. For få epoker vil på den andre siden gi en undertrent modell som verken klarer å tilpasse sensordata eller de fysiske betingelsene tilstrekkelig. Antall epoker påvirker også den totale beregningstiden, siden hver epoke går gjennom tapsfunksjonen og en oppdatering av parametrene. Valg av antall epoker innebærer derfor et kompromiss mellom beregningstid og modellkvalitet, slik at utgangspunktet på 5000 epoker kan være et godt alternativ.

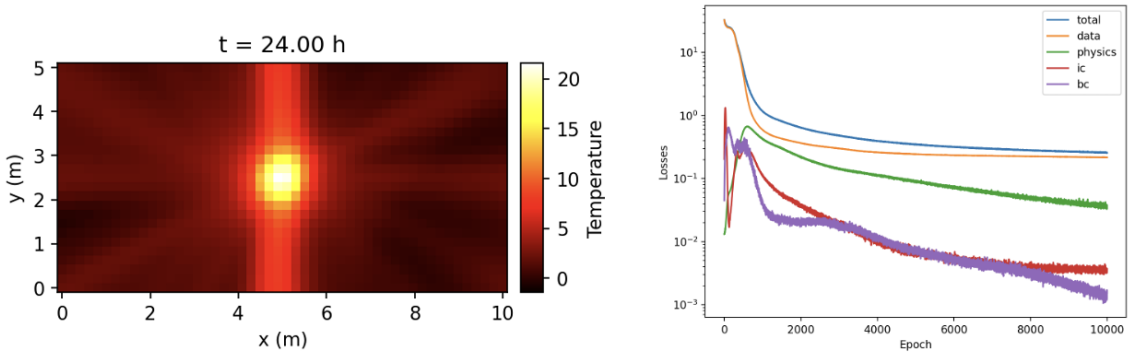


Figure 20: Viser temperaturfordelingen og tapsfunksjonene til PINN-modellen etter å ha økt til 10 000 epoker. Grafen til høyre indikerer at modellen vil fortsette å tilpasse seg fysikk-, initial- og randbetingelsene, slik at den til slutt blir overtilpasset disse betingelsene, som vist i temperaturfordelingen etter 24 timer til venstre.

3.2 Oppgave 5.6

	Fysiske parametre	Sanne verdier
α $[\mathrm{m}^2\,\mathrm{s}^{-1}]$	≈ 0.0288	0.25
k $[W\,m^{-2}\,K^{-1}]$	≈ 0.233	0.04
h $[W\,m^{-1}\,K^{-1}]$	≈ 0.0997	0.00
P $[\mathrm{K}\,\mathrm{s}^{-1}]$	≈ 1.934	15

Table 1: Tabell over fysiske og teoretiske verdier for parametre. Parameteren α er rommets termiske diffusivitet, k er veggenes varmeledningsevne, h er varmeoverføringskoeffisienten og P er styrken fra varmekilden.

Tabell 1 viser de fysiske parameterne estimert av PINN-modellen sammenlignet med de sanne verdiene brukt i den numeriske løseren. Resultatene fra seksjon 3.1 viser at PINN klarer å lære et temperaturfelt som i stor grad samsvarer med den numeriske løsningen, men at de estimerte parameterverdiene avviker betydelig fra de teoretiske. Dette indikerer at PINN først og fremst lærer parametere som gir riktig temperaturutvikling i rommet, i stedet for å gjenskape de fysiske konstantene fra modellen.

Den termiske diffusiviteten α estimeres til en betydelig lavere verdi enn den teoretiske. Samtidig estimeres både varmeledningsevnen k og varmeovergangskoeffisienten h til verdier som er høyere enn de teoretiske, mens styrken til varmekilden P blir lavere enn verdien brukt i den numeriske løseren. Disse avvikene må sees i sammenheng, ettersom parameterne i varmeligningen ikke er uavhengige når de estimeres fra sensordata. Flere kombinasjoner av α, k, h og P kan gi tilnærmet samme temperaturfelt i de områdene der målingene finnes, og PINN vil derfor konvergere mot en av mange mulige løsninger som minimerer den totale tapsfunksjonen.

Parameterestimatene er også sterkt avhengige av sensorenes plassering. Parameterne k og h har størst innvirkning på temperaturutviklingen nær veggene, mens P påvirker temperaturen i nærheten av varmekilden. Når sensordataene er få eller dårlig plassert, inneholder de begrenset informasjon om for eksempel varmetapet til omgivelsene. For å kompensere for dette kan PINN øke både varmeledning og varmetap samtidig, mens styrken til varmekilden reduseres, uten at dette gir store utslag på temperaturprediksjonene. Resultatet blir parametre som samsvarer med observasjonene og temperaturfeltet, men som ikke stemmer overens med de sanne teoretiske verdiene.

Til tross for at sensor- og støyavhengigheten til parameterne, ble det observert at selv ved å øke antall sensorer og redusere støynivået i målingene, i config.yaml filen, endret de estimerte fysiske parameterne seg i liten grad. Dette støtter antakelsen om at PINN i dette tilfellet trenes til å finne en kombinasjon av parametere som gir lav tapsverdi, fremfor å identifisere de teoretisk korrekte parameterverdiene. Det finnes altså flere sammensetninger av parametre som kan simulere den samme temperaturfordelingen.

3.3 Oppgave 5.7

Tapsfunksjonen brukt til å trene PINN er oppgitt i ligning (21). Denne ligningen kombineres av tapskomponenetene $\mathcal{L}$ multiplisert med deres respektive "vekter", λ.

$$\mathcal{L}(\theta) = \lambda_{data}\mathcal{L}_{data}(\theta_{NN}) + \lambda_{ic}\mathcal{L}_{ic}(\theta_{NN}) + \lambda_{bc}\mathcal{L}_{bc}(\theta) + \lambda_{physics}\mathcal{L}_{physics}(\theta) \tag{21}$$

Som navnet tilsier, er hensikten med vektene å bestemme hvor mye hver enkelt tapskomponent vil vektlegges i treningen av PINNs ved å skalere gradienten. Hvordan de velges og balanseres i forhold til hverandre, har stor innvirkning på systemet. Om en av disse vektene har en forhøyet verdi som overgår de resterende, kan dette gå på bekostning av fysikken, data eller randbetingelser. I treningen av PINN-en tar tapsfunksjonen inn sensordata, initial- og randbetingelser, i tillegg til fysiske lover. Dersom λ_{data} blir for stor, vil det føre til at den interpolerer svært godt. Den vil derimot ta mindre hensyn til fysikken og ligne mer på et nevralt nettverk. Vektlegges initial- eller randbetingelsene høyere vil systemet prioritere å trene på å løse dette godt. Det vil samtidig lære det indre domenet sakte, som resulterer i en dårlig global løsning og et ufleksibelt system. Til sist vil en vektlegging av $\lambda_{physics}$ gi en modell som samsvarer godt med PDE, men dårlig med sensordata. Den vil også bryte med initial- og randbetingelser for å prioritere å tilfredsstille PDE. Dermed vil også parametre som α, h, k, og P justeres etter løsning av PDE i stedet for å ta utgangspunkt i virkeligheten.

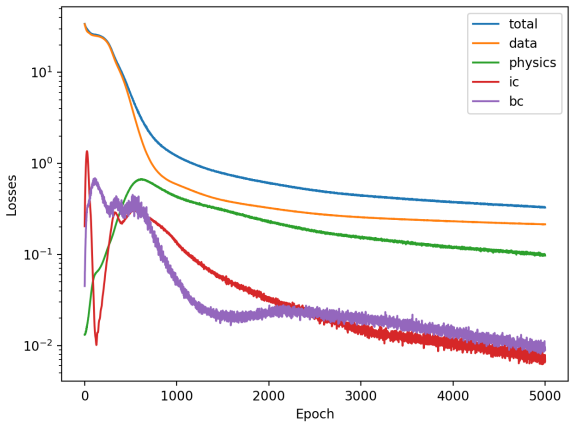


Figure 21: Viser utvikling av minimering av tap for PINN over 5000 epoker.

Ingen av tapsfunksjonene klarer alene å trene PINN optimalt. Derfor bør vektene være balanserte under trening av PINN slik at ingen av tapsleddene blir i overkant dominerende. Avhengig av systemet er det ikke nødvendigvis riktig at vektene skal være helt like, men det er et godt utgangspunkt for å starte. Ved å først trene et system med like vekter vil det deretter være mulig å skalere de opp og ned etter hvordan de ulike tapene observeres for å optimalisere systemet. Fra figur 21 observeres det

at tapene for initial- og randbetingelser allerede er svært lave. Da vil neste steg være å undersøke hvordan $\lambda_{physics}$ og λ_{data} kan justeres for å minimere tapene.

Ved å oppjustere $\lambda_{physics}$ fra verdien 1.0 til 5.0 minker som forventet tapet knyttet til fysikkleddet $\mathcal{L}_{physics}$, som vist i figur 22. Som resultat av dette observeres en svak økning for tapsfunksjonen til sensordata i tillegg til det totale tapet. Dette skyldes at modellen i større grad prioriterer de fysiske ligningene på bekostning av tilpasning til sensordata. Som vist i figur 23 fremstår temperaturutviklingen som mindre jevn enn for lik vekting av tapskomponentene. Dette kan skyldes at modellen gjør kompromisser mellom data- og fysikkleddene, noe som gir lokale ujevnheter. Temperaturprediksjonen ligner prediksjonen i figur 20 hvor antall epoker økes til 10000. Som observert i plottet for tapsfunksjonen minimeres $\mathcal{L}_{physics}$ sammen med initial- og randbetingelsene, mens det er liten endring i tapet for data i likhet med for økt vekting av $\lambda_{physics}$.

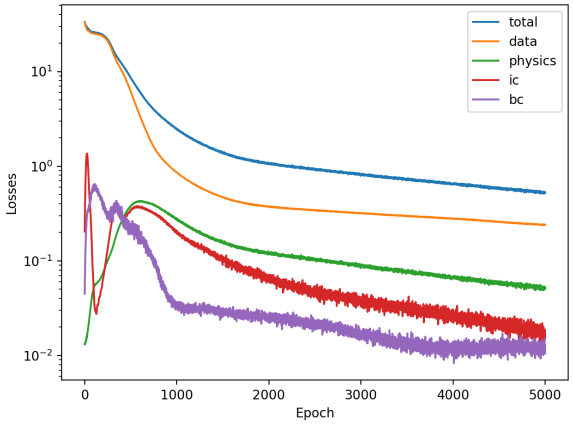


Figure 22: Viser minimering av tap over 5000 epoker. Manuell justering av $\lambda_{physics} = 5.0$. Resterende vekter satt til lik verdi, $\lambda_{data} = \lambda_{ic} = \lambda_{bc} = 1.0$

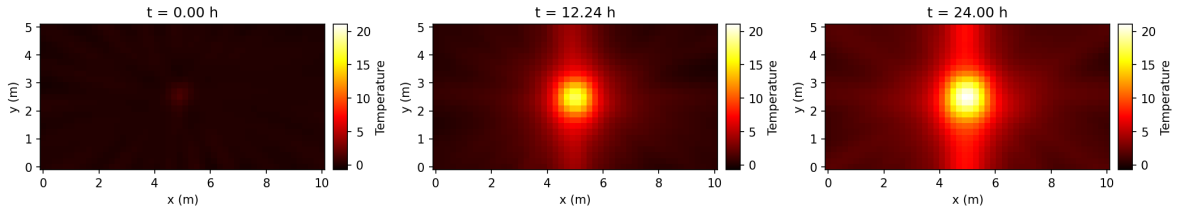


Figure 23: Varmeutvikling over 24 timer med $\lambda_{physics} = 5.0$ (Ligner på den fra 5.5, gir mening siden begge tilpasser seg fysikken

Samme økning for λ_{data} resulterer i en temperaturspredning som i større grad ligner det nevrale nettverket som vist i figur 25. I likhet med det nevrale nettverket vil treningen vektlegge sensordata på bekostning av fysiske lover. Selv om $\mathcal{L}_{data}$ har en liten reduksjon som vist i figur 24, observeres det en markant økning for det totale tapet. Overvekting av data kan tvinge frem løsninger som er inkompatible med fysikken, mens overvektingen av fysikk-leddet i større grad gir løsninger som er forenlige med data. Dette indikerer at data-leddet i større grad er i konflikt ovenfor de andre tapskomponentene. En økning av $\lambda_{physics}$ vil altså ikke ha like stor innvirkning som en tilsvarende økning av λ_{data}, men det vil likevel være mest hensiktsmessig å finne en balanse.

En manuell justering av vektene er en tungvint fremgangsmåte i undersøkelse av tap. Selv om vektene er like, kan gradientene til tapskomponenetene ha en svært ulik størrelse. Dermed ville en alternativ løsning være å justere vektene automatisk under trening. For å utføre dette må vektene være trenbare. Dermed vil det være mulig å innføre en automatisk læringsprosess under treningen, såkalte "self-adapting PINNs" [1]. Økning av et spesielt tap vil lede til økning av relatert vekt for

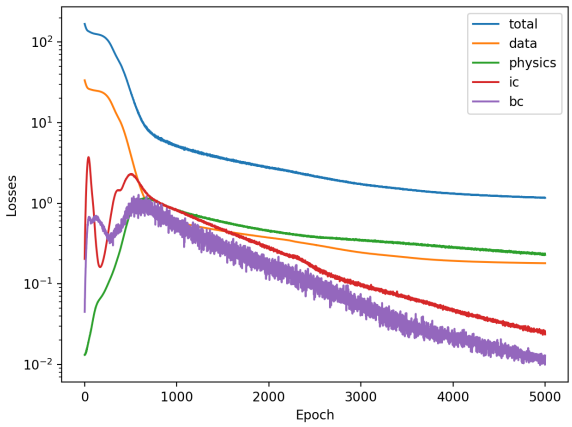


Figure 24: Viser minimering av tap over 5000 epoker. Manuell justering av $\lambda_{data} = 5.0$. Resterende vekter satt til lik verdi, $\lambda_{physics} = \lambda_{ic} = \lambda_{bc} = 1.0$

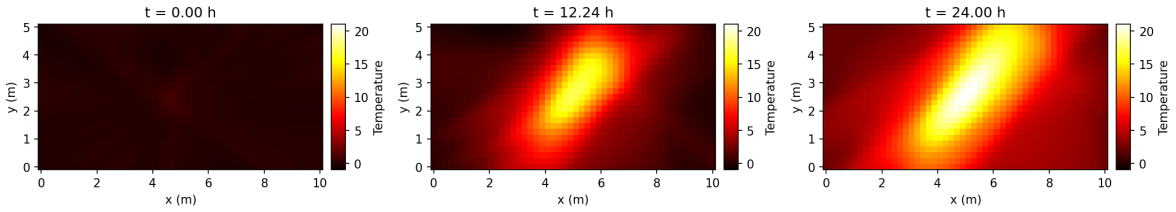


Figure 25: Varmeutvikling PINN over 24 timer med $\lambda_{data} = 5.0$

å minimere, og gradientbidragene balanseres. Dette kan være vanskelig å innføre ettersom vektene trenes basert på tap, samtidig som de påvirker gradientene. Optimeringsprosessen vil være sensitiv og kan bli ustabil eller krevende å implementere, men til gjengjeld gir den en mer generaliserbar løsning.

3.4 Oppgave 5.8

De tradisjonelle numeriske metodene klarer å beregne temperaturfeltet på en effektiv og realistisk måte, både for varmeledning og tilsvarende for PDE. Når fysiske parametre og nødvendig informasjon er kjent, vil den numeriske metoden være et godt og effektivt alternativ for å beregne et temperaturfelt med høy nøyaktighet. PINN vil i en slik situasjon sjelden være et bedre valg, og det vil være naturlig å spørre seg hvorfor bruke PINNs når den numeriske løseren allerede klarer å beregne temperaturfelt med høy presisjon. Fordelene med PINN ligger ikke nødvendigvis i løsningshastighet, men i integrering av informasjon. For et system med begrenset data eller inverse problemer, og ukjente fysiske parametre, viser PINN seg å være svært nyttig. Dette er fordi modellen klarer å kombinere fysiske lover, rand- og initialbetingelser, og måledata i en og samme modell. Dermed kreves mindre treningsdata, og PINN vil klare å estimere bedre for støyete eller ufullstendig data.

I forskningsartikkelen, "Physics-Informed Neural Networks for predicting indoor temperature and cooling demand in historic building", sammenlignes PINNs med nevrale nettverk for temperaturprediksjon i historiske bygninger [4]. En av problemstillingene som tas opp angående NN er hvordan de fungerer i ikke-stasjonære forhold. Med endring i temperatur og klima krever det kontinuerlig overvåkning av systemet for å forsikre seg at prediksjonene ikke er ugyldige. Dette vil bli en større utfordring i en fremtid som bringer mer ekstremvær og endrende klima. Det er slike problemstillinger som gjør at det trengs en hybridløsning som tar inn sensordata og samtidig tar hensyn til fysikken, altså PINN.

En tydelig ulempe er derimot at PINNs kan være både vanskelige å trene og beregningstunge. De ulike tapskomponentene til $\mathcal{L}(\theta)$ har ulik gradientstyrke, altså vil vektene være sensitive og modellen

kan være numerisk ustabil. Til sammenligning med et nevralt nettverk vil PINNs være mer komplekse å implementere. Det nevrale nettverket krever kun sensordata, og forholder seg ikke til hverken fysikk eller sensitive gradienter og tapskomponenter. Det gjør at et NN er enkel å trene, men dårlig å generaliserer når det er mangel på treningsdata. Med få sensorer vil det være foretrukket med PINN i stedet for NN som fremstilt i figur 15 kontra figur 4. Dersom det er flere sensorer, klarer NN bedre å fremstille varmespredning slik som observert i figur 7. Figur 17 som fremstiller PINN med likt antall sensorer er betydelig bedre, men det viser at det også er mulig å fremstilles en brukbar varmespredning med NN dersom det er ønskelig med et mindre krevende treningssystem. Det er også verdt å nevne at simuleringen har en svært forenklet PINN-modell med lite sensordata, slik at i et mer realistisk tilfelle vil en PINN være langt mer omfattende og tidskrevende, som vil være en utfordring.

Det er viktig å poengtere at hensikten med PINN ikke er ment for å erstatte numeriske metoder. Begge har sine fordeler og ulemper, og anvendes best i ulike situasjoner. For en problemstilling med kjente randbetingelser og geometri vil numerisk metode være mest effektivt. PINNs er et mer komplekst, men også fleksibelt, alternativ for situasjoner hvor mangel på måledata må kombineres med fysiske lover i samme modell. NN er til slutt et godt alternativ hvis det finner veldig god data.

3.5 Oppgave 5.9

Resultatene fra arbeidet med PINN-modellen viser at temperaturutviklingen i rommet kan rekonstrueres på en måte som ligner den numeriske løseren, men at de estimerte fysiske parameterne avviker fra de teoretiske verdiene i tabell 1. Det indikerer at PINN prøver å reprodusere temperaturfeltet ved å kompensere gjennom det nevrale nettverket, i stedet for å tilpasse seg de riktige fysiske parameterne. For videre arbeid er det flere forbedringer som kan bidra til å gjøre modellen mer realistisk, i tillegg til å produsere resultater som har større sammenheng med de fysiske lovene.

For å få en modell med rimelige fysiske parameterestimat, er en mulig forbedring å tvinge de estimerte fysiske parameterne mot mer rimelige verdier. Dette kan gjøres ved å «straffe» urealistiske sammensetninger av parametre gjennom å justere tapsfunksjonene. For eksempel kan fysikkleddet vektes høyere, eller så kan det innføres en regularisering som begrenser α, k, h og P til intervaller rundt de teoretiske verdiene som ønskes. Denne løsningen kan hindre at modellen justerer parameterne kun for å passe temperaturfordelingen, uten hensyn til fysikken.

Videre kan dette problemet løses ved å teste ut tidsavhengige eller romavhengige fysiske parametere. I utgangspunktet antar modellen at disse parameterne er konstante, men i realiteten kan de variere både med tid og posisjon. Dette kan komme av blant annet endring i utetemperatur eller at rommet har ulik isolasjonsevne. Ved å gjøre parameterne til funksjoner av tid eller rom, eller eventuelt til egne nevrale nettverk, vil det gi en mer realistisk modell som bedre tilpasser seg større og mer komplekse rom, i tillegg til å redusere behovet for kompensasjon i PINN.

En annen mulig forbedring, er å representere varmekilden som et eget nevralt nettverk. I den nåværende implementasjonen er varmekilden definert på forhånd med en bestemt størrelse og styrke, noe som kan være en forenkling av den faktiske situasjonen. Ved å la et nevralt nettverk modellere varmekilden direkte, kan både temperaturfordelingen og styrken læres fra sensordata. Dette gir modellen større fleksibilitet og kan redusere behovet for at andre fysiske parametere tilpasses urealistisk for å kompensere for en feilaktig antatt varmekilde.

Et mer avansert forbedringstiltak er å benytte Hamiltonian Neural Networks [7], for å ta større hensyn til energikonservering. I stedet for å lære systemets dynamikk direkte, parametriserer en slik modell Hamilton-funksjonen ved hjelp av et nevralt nettverk, som kan lære denne fra sensordata. Energibevaring er da innebygd i modellstrukturen, siden tidsutviklingen følger Hamiltons ligninger, og modellen tar dermed hensyn til energikonservering uten behov for eksplisitte straffefunksjoner. Etter at modellen har lært fra dataene, kan den gi mer fysisk konsistente og stabile prediksjoner over tid. Ved å beskrive systemets dynamikk gjennom en Hamilton-funksjon tvinges modellen til å følge hamiltons ligninger, noe som bidrar til bedre bevaring av energi-lignende størrelser sammenlignet med standard nevrale nettverk. Det kan gi mer realistiske parameterestimater, særlig i situasjoner med begrenset datatilgang. [7]

Videre vil det være relevant å se nærmere på treningsstrategien til PINN-modellen. I filen train.py er funksjonene train_nn() og train_pinn() ansvarlige for treningen av henholdsvis det nevrale nettverket og PINN. I disse implementasjonene utføres det i praksis ett optimaliseringssteg per epoke. Slik som oppgaveteksten foreslår, vil en mer effektiv tilnærming være å skille mellom epoker, batches og iterasjoner. En epoke er en gjennomgang av hele datasettet. Her kan treningen i hver epoke deles inn i batches, der det utføres en iterasjon, altså et adam_step per batch, samtidig som nye punkter for fysikkleddet, initialbetingelser og randbetingelser trekkes stokastisk for PINN. Dette gir flere gradientoppdateringer per epoke, bedre utnyttelse av data og mer stabil konvergens, spesielt for større og mer komplekse datasett.[1]

I artikkelen "Physics-informed neural networks for modeling dynamic linear elasticity" [5] foreslås det at treningen av PINN kan deles inn i flere faser med ulik læringsrate som gradvis reduseres. På denne måten kan modellen først lære de grove strukturene i temperaturfeltet før løsningen finjusteres i senere faser. Som diskutert tidligere i del 3.3 har også vektene i tapsfunksjonen stor innvirkning på resultatene, og en videre utvikling vil derfor være å gjøre disse vektene trenbare, eller å justere dem mellom treningsfaser slik som for læringsraten.

I den nåværende modellen trenes én PINN for hver enkelt situasjon, hvor nye randbetingelser, varmekilder og fysiske parametere krever ny trening fra bunnen av. Dette er tungvint og lite skalerbart. Å amortisere modellen kan gjøre den raskere i bruk og mer egnet for praktiske anvendelser. Amorterisering handler om å trene én modell på mange ulike situasjoner samtidig. Modellen lærer da en direkte sammenheng mellom problemets parametere og det tilhørende temperaturfeltet. Etter trening kan nye situasjoner vurderes direkte, uten videre optimalisering. For å bruke amortisering må problemparametre gis som egne input i nettverket. Modellen må deretter trenes på et variert sett av situasjoner. Som konsekvens av dette blir modellen mer fleksibel og anvendelig i praksis.

Til slutt er både NN- og PINN-modellen sterkt påvirket av kvaliteten på sensordata, slik at å flere sensorer med bedre dekning og mindre støy vil forbedre modellene. Dette kan igjen ha en påvirkning på de estimerte parameterne dersom modellen tvinges til å ta mer hensyn til dette. Sensorplassering. Som oppgaveteksten foreslår kan varmekilden gjøres mer fleksibel ved å modellere den som et nevralt nettverk, slik at modellen kan lære direkte fra data, og igjen kunne gi oss riktige fysiske parametre.

For å konkludere viser resultatene at PINN-modellen kan rekonstruere temperaturfeltet godt, men at estimerte fysiske parametre kan bli urealistiske. Det viser til at modellen delvis kompenserer gjennom det nevrale nettverket, i stedet for å tilpasse seg de riktige fysiske parameterne. Strengere regularisering av fysiske parametre, mer fleksible nevrale nettverk for varmekilder og amortisering av treningen kan gi mer konsistente og fysisk realistiske resultater.

References

[1] L. D. McClenny and U. M. Braga-Neto, "Self-adaptive physics-informed neural networks," *Journal of Computational Physics*, volume. 474, p. 111722, 2023, doi:10.1016/j.jcp.2022.111722.

[2] Elling Svee, *Fysikk-informerte nevrale nettverk – Prosjekt 1 i TMA4320*, NTNU, 2025.

[3] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, Philadelphia, 2003.

[4] S. Semeraro, F. Vecchi, R. Stasi, and U. Berardi, "Physics-Informed Neural Networks for predicting indoor temperature and cooling demand in historic buildings," *Journal of Computational Physics*, volume. 115, p. 114392, 2025, doi:10.1016/j.jobe.2025.114392.

[5] V. Kag and V. Gopinath, "Physics-informed neural networks for modeling dynamic linear elasticity," *arXiv preprint arXiv:2312.15175v2*, 2024.

[6] DNV and Norsk Industri, "Energy Transition Norway 2022: A National Forecast to 2050," *DNV Report*, 2022.

[7] Greydanus, S., "Hamiltonian Neural Networks," *Blog post*, 2019. https://greydanus.github.io/2019/05/15/hamiltonian-nns/